RELATION SHIP

21/02/202

Public

It is the "connection" between two Objects

~~Vehicle Car is a four wheeler vehicle~~

Car is a four wheeler vehicle

Marker has a cap

Heart has blood vessels.

Apple is a fruit.

Mango tree has a Mango

~~Or~~ Bank has Account holders.

The relationship between objects can be achieved with the help of following ways:

1) has - A - relationship.

2) is - A - relationship

has - A - relationship':

has - A - relationship is just similar to relationship between a whole object and a part of it.

Eg: Qspider has students

Bank has Accounts

Books has pages

Car has Engine.

NOTE: In Java, we can achieve has-A-relationship between two objects by creating a reference of one object inside another object.

▶ COMPOSITION :

The process of one object having a reference of another object is known as Composition.

Composition can be achieved with the help of below designed techniques:-

1) Early binding

2) Lazy binding

▶ EARLY BINDING:

Early binding is a design technique in wh the composed object has to be created during th loading process of whole object.

Eg: Car & Engine

Book & pages

Employee & EmployeeID

Account & Account Number.

Let us consider the example of Car & Engine as shown below:-

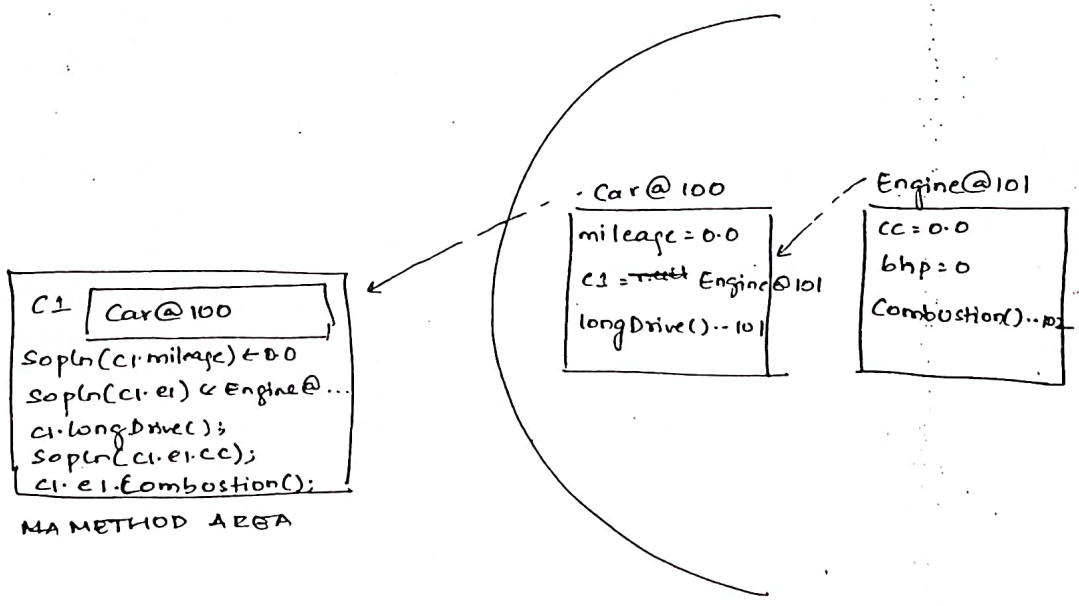
```
class Engine
{
double cc;
int bhp;
void Combustion()
{
System.out.println("Power Generated");
}
}
```

```
class Car
{
    double mileage;
    Engine e1 = new Engine();
    void longDrive()
    {
    System.out.println("Nice Drive!");
    }
}

class Driver1
{
public static void main (String[] args)
{
Car c1 = new Car();
System.out.println(c1.mileage);
System.out.println(c1.e1);
c1.longDrive();
System.out.println(c1.e1.cc);
~~by~~c1.c1.combustion();
}
}
```

Op:

```
0.0
Engine@101
Nice Drive!
0.0
Power Generated.
```



Assignment:

Q. Design the blueprints for Mobile & Battery. Establish the relationship b/w them using suitable design technique.

```
class Battery
{
int mAh;
void charging()
{
System.out.println("Charging done");
}
}
class Mobile
{
double price;
String processor;
Battery b1 = new Battery();
~~System.out.println~~
void watchVideo()
{
System.out.println("Video played!");
}
}
class Driver2
{
public static void main(String[] args)
{
Mobile m1 = new Mobile();
System.out.println(m1.price);
System.out.println(m1.processor);
m1.watchVideo();
System.out.println(m1.b1.mAh);
m1.b1.charging();
}
}
```

LAZY BINDING:

It is a design technique in which the composed object should not be created during the loading process of whole object, but it has to be created only with the help of behaviours (method of same object).

Eg: ~~class Stud~~ Ospiders has student

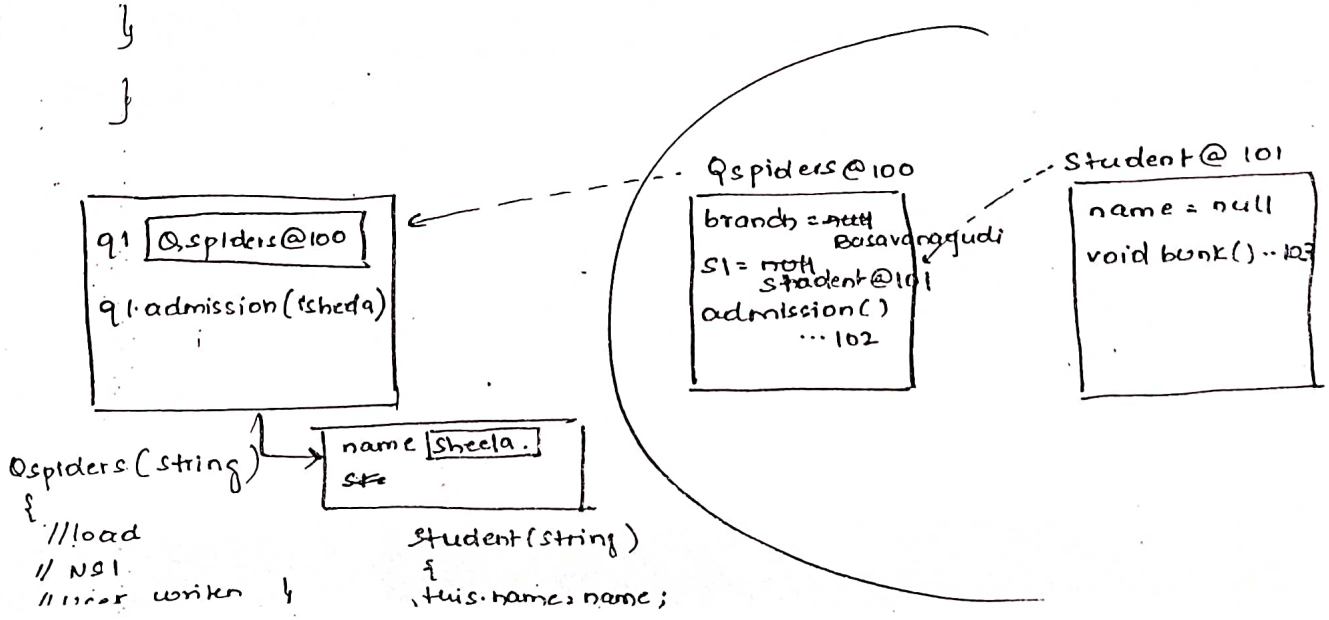
Banks has Accocent

Mangotree has Mango.

Company has Employee.

```
class student
{
String name;
student ()
 {
 }
student (String name)
 {
this.name= name;
 }
 void bank ()
 {
 So.plu ("loose knowledge");
 }
}
```

```
class Qspiders
{
String branch;
Qspiders()
{
}
Qspiders(String branch)
{
this.branch= branch;
}
Student s1;
void admission(String name)
{
s1= new Student(name);
Sopln(name+"is admitted");
}
}
class Driver2
{
public static void main(String[] args)
{
Qspiders q1 = new Qspiders("Basaranagudi");
q1.admission("Sheela");
}
}
```



In lazy binding design technique, the composed object will not be created during the loading process of whole object. Instead, it will be created only with the help of behaviour of same object.

22/02/2022

▶ Is-A- Relationship:

It is just similar to a relationship between parent and a child where the child aquires all the states and behaviours of parent.

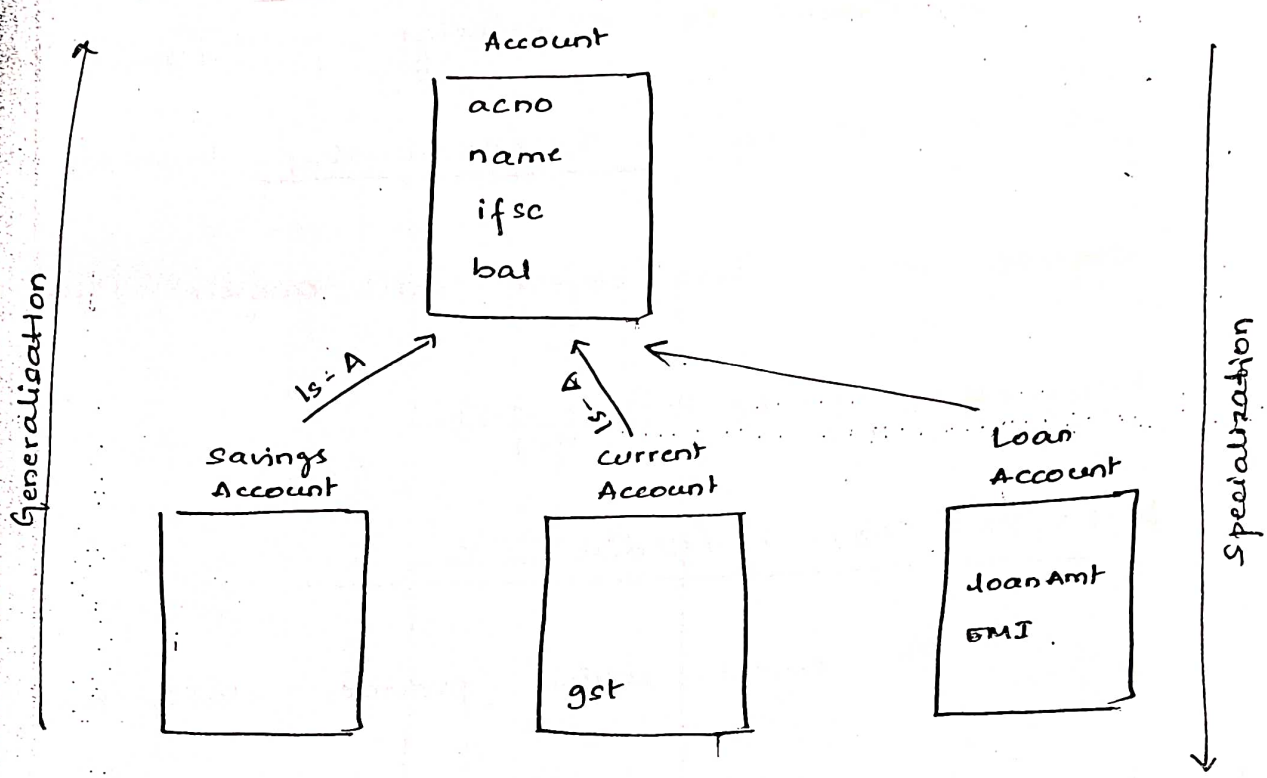
Eg: Banking is a Business.

Food is a Basic Element of life

Java is a programming language.

Savings account is an account.

purpose of Is-A- Relationship :



The main purpose is to achieve two important things. They are

1) Generalisation (Bottom-to-Top Approach)

2) Specialization (Top to Bottom Approach)

NOTE : Is-A- Relationship can be achieved using Inheritance in java.

INHERITANCE

Inheritance is a design principle which is used to achieve Is-A- relationship between two Objects. where one object will acquire states & behaviours of another object.

➤ PARENT OBJECT / CLASS:

An object which provides states & behaviours to another object is known as PARENT OBJECT.

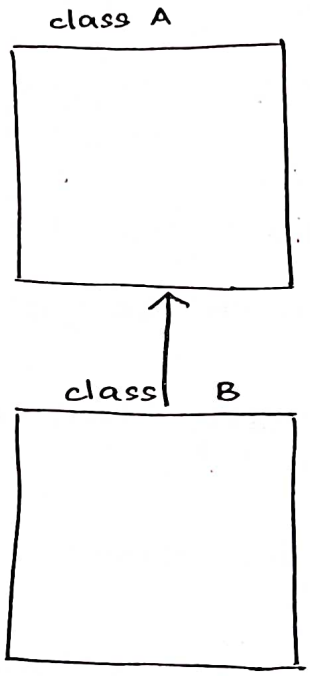
Parent Object is also known as SUPER OBJECT, BASE Object, GENERALISED object

➤ CHILD OBJECT / CLASS:

A class which recieves states & behaviours from another class is knocon as CHILD CLASS.

Child class is also known as Sub class, Derived class & Specialigeol class.

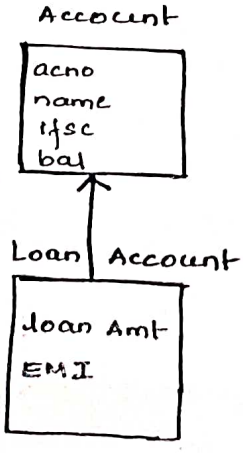
Inheritance can be diagramatically represented as below:



In the above Ex, class A is parent class & class B is child class.

NOTE: In the solid Arrow, the head of the Arrow must always be pointing towards parent class whereas the Tail of the Arrow must always be pointing to the child class.

Eg:



Let us consider Accounts class which has following states:

1) Account Number
2) Account name
3) IFSC code
4) Balance

The Loan account which is the child class of Account class will have the states declared inside it ie

1) Loan Amount
2) EMI ; as well as the states inherited from Accounts class. (Account number, Account name, IFSC, balance).

NOTE: In Java, we can achieve Inheritance using following keywords:

1) extends
2) implements

INHERITANCE W.R.T STATICS:

Static members are inherited

NOTE:

i) In the loading process with respect to Inheritance Super class will be loaded first & then the Sub class will be loaded.

ii) Sub class's CSA will have a built-in reference to the Super class CSA. Therefore we can use:

i) The members of Super class can be used/accessed directly inside Sub-class.

ii) We can also use Super class members with the help of Sub-class reference (its name)

Eg for <i>:

```
class A
{
static int i = 20;
}
class B extends A
{
public static void main (String [] args)
{
System.out.println (i);
}
}
```

o/p:

The main method of class can also be inherited.

Eg:
```
class B
{
public static void main (string [] args)
{
    System.out.println ("from main");
}
}
class A extends B
{
}
```

Op:

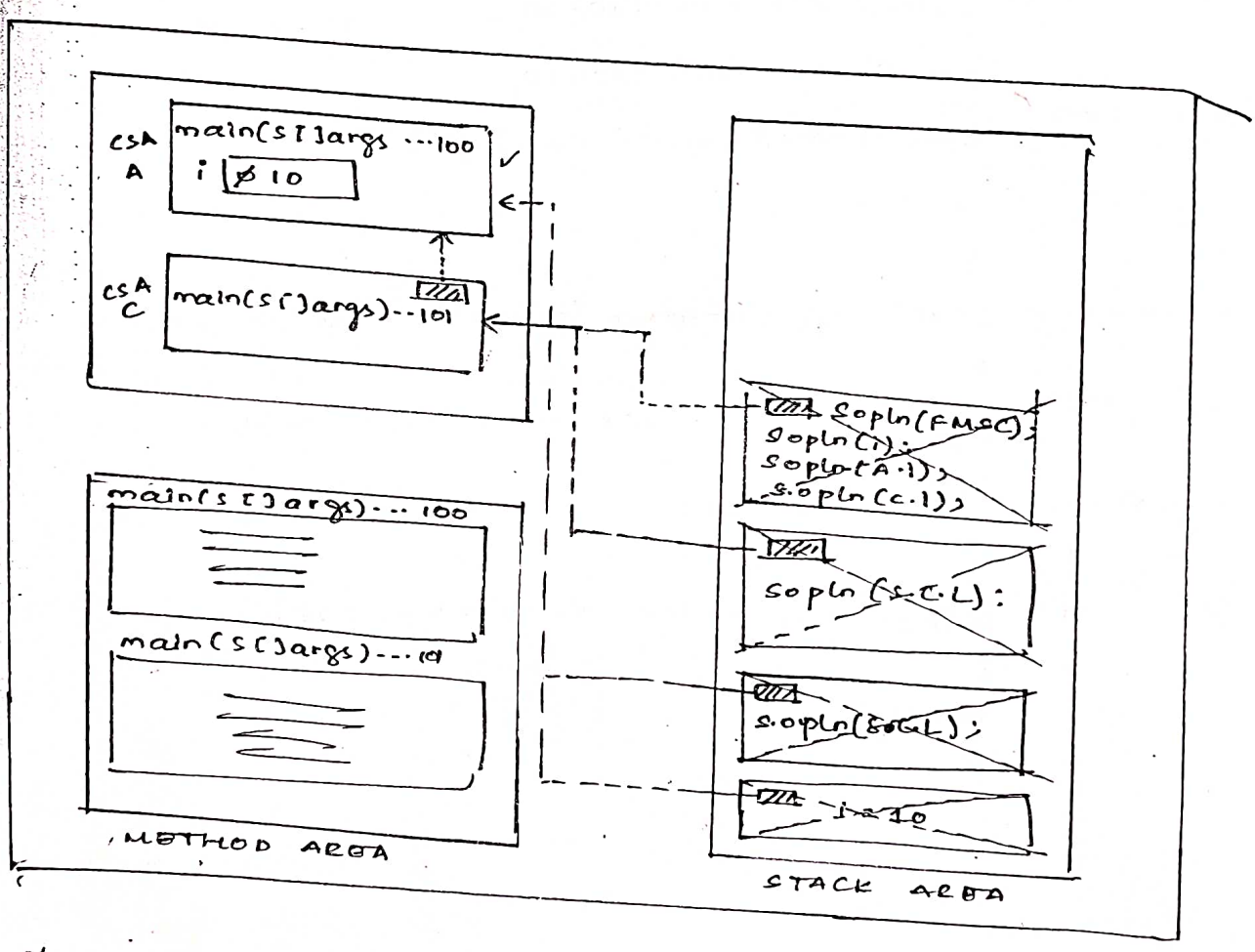
from main

Example for proving superclass gets loaded first as well as <if>

```
class A
{
static int i=10;
static
 {
  System.out.println ("super class Loading");
 }
public static void main (String [] args)
{
 System.out.println ("from super class");
}
}
class C extends A
{
static
 {
    System.out.println ("sub class Loading");
 }
```

```
public static void main (String [] args)
{
System.out.println ("From main of sub class");
System.out.println (i);
System.out.println (A.i);
System.out.println (c.i);
}
}
```

STACK TRACING :



Op :

Super class Loading

Sub class Loading

From main of sub class

10
10
10

```
class Calculator1
{
public static int add (int a, int b)
{
return a+b;
}
public static int sub (int a, int b)
{
return a-b;
}
}
class Driver1
{
public static void main (String[] args)
{
Calculator1.add (10,20);
Calculator1.sub (30,10);
Calculator1.mul (10,2); //CTE
}
}
```

called as "patch"

```
class Calculator2 extends Calculator1
{
public static int mul (int a, int b)
{
return a*b;
}
public static int div (int a, int b)
{
return a/b;
}
}
class Driver2
{
public static void main (String[] args)
{
Calculator2.mul (10,2);
Calculator2.div (20,4);
Calculator2.add (10,20);
Calculator2.sub (50,10);
}}
```

Using subclass reference we can use members of both sub class & super class. But members of sub class cannot be accessed using super class Reference.

NOTE: >In java, it is mandatory for each & every class created must & should extend to another class.

---

2) If the user is not extending the class to any other class, by default compiler will extend it to a default supermost class of java known as Object class.

---

3) Object class is a built in class which is present inside a built-in-package known as "java.lang" Package.

Q. WAJP to print "smith is a Great batsman". using constructor. where in ~~such than~~ constructor can have only one word.

```
class print
{
 print( String a)
 { System.out.println("a");
 }
 print (string a, string b)
 {
 this (a);
 System.out.println ("b");
 }
 print (string a, string b, string c)
 {
 this (a, b);
 System.out.println (c);
 }
 print (string a, string b, string c, string d)
 {
 this (a, b, c);
 System.out.println (d);
 }
 print (string a, string b, string c, string d, string e)
 {
 this (a, b, c, d);
 System.out.println (e);
 }
}
class Driver
{
public static void main (String [] args)
{
new print("smith", "is", "a", "Great", "batsman");
}
}
```

CONSTRUCTOR CHAINING

The process where one constructor calls another constructor is known as constructor chaining

The constructor chaining can be achieved using following ways:

1) this call statement → this()

2) super call statement → super()

➤ this() :

this() is used to achieve constructor chaining within the same class.

➤ Rules for this call statement :

1) "this()" should be used "only inside constructors"

2) this() should be the first instruction inside constructor

3) Recursive invocation of this() is not allowed

(OR)

this() should not call the constructor recursively

4) If a class is having 'n' no of constructors, then we can have only (n-1) no of this()
inside the class.

NOTE :

1) If a constructor has this() used inside it, the compiler will not add the predefined instructions inside it ie Load instruction and Instruction to load Non-static initializers inside constructor.

2) this() will "never be added by the compiler", it is always user defined statements.

Eg:

```
class Tv
{
 int tid;
 String brand;
 double price;
 TV()
 {
 }

 TV(int tid)
 {
  this();
  this.tid= tid;
 }
```

Demo1.java

```
class Demo1
{
Demo()
 {

 }
 Demo(int i)
 {
 this();
 }
 {
  sopln("IIB");
 }
}
```

javac Demo1.java →

Demo1.class

```
class Demo1
{
 Demo()
 {
 // Load instructions
 // NSI
  {
   sopln("IIB");
  }
 //User written inst
 }
 Demo(int i)
 {
  this();
  //User written inst
 }
}
```

INHERITANCE W.R.T NON-STATICS :

The non-static members of the class will be Inherited.

NOTE :

1) With the help of Sub-class reference, we can access the members of Super class & Sub class.

Eg:

```
class A
{
   int i=10;
}
class B extends A
{
   int j=20;
}
class Driver3
{
   public static void main(String[] args)
   {
      B obj = new B();
      System.out.println(obj.j); //20
      System.out.println(obj.i); //10
   }
}
```

(2) With the help of super-class members, we can access only the members of super-class, we will not be able to access the members of sub-class.

Ex:
```
class A
{
 int i=10;
}
class B extends A
{
 int j=20;
}
class Driver2
{
 public static void main (String[] args)
 {
   A obj = new A();
   System.out.println(obj.i); //10
   System.out.println(obj.j); //CTE
 }
}
```

▶ LOADING PROCESS OF OBJECT WITH RESPECT TO INHERITANCE :-

※ Whenever we create a subclass object, it will have :

1) The members of superclass.

2) The member of subclass.

This is done with the help of constructor chaining using SUPER CALL STATETMENT [super()]

▶ super() :.

1) super() is used to call the constructor of super class from sub class.

※ 2) The super() will be added to each & every constructor inside which we dont have this().

Eg:

~~clas~~ A.java

```
class A
{


}
```

javac A.java →

A.class

```
class A extends Object
{
  A(){
  //super()
  // Load instructions
  // NSI
  }
}
```

RULES FOR super():

1) super call statement has to be written inside constructors only.

2) It should always be the first instruction inside the constructor.

3) We cannot have both super() & this() within same constructor. ... as well as .....

4) If we have 'n' no of constructors we ~~to~~ can have 'n' no of super ~~class~~ call. statements as there is no problem of Recursion here.

```
class x
{
  x() { }
 x(int i) {
  this();
 }
 x(float i) {
  super();

 }
}
```

javac x.java →

x.class

```
class x extends Object
{
 x()
 { super();
 // load instructions
 // NSI
 // user written instructions
 }
 x(int i) {
  this();
  //user written instructions;
 }
 x(float i) {
  super();
//user written instructions
} // load instructions
  // NSI
  // user written instructions
```

For the below program write the following

1) The order of Invocation of constructor

2) The order of Execution of constructor

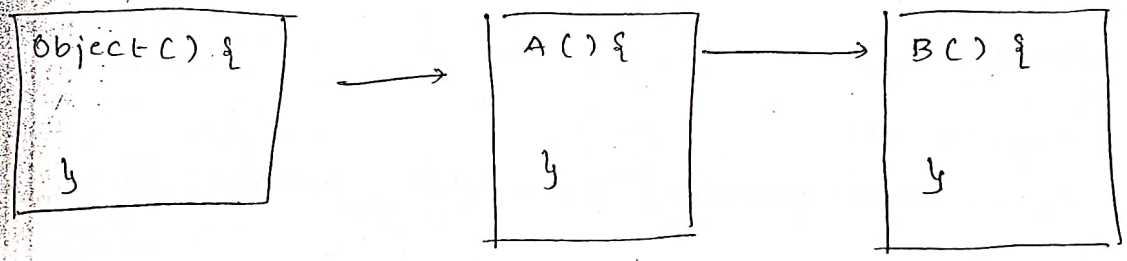
3) Output of the program.

```
class A
{
    {
     System.out.println("From class A IIB-1");
    }
    A()
     {
       System.out.println("From classA()");
     }
}
class B extends A
{
    {
     System.out.println("From class B IIB-2");
    }
    B()
     {
      System.out.println("From class B()");
     }
}
class Driver4
{
  public static void main(String[] args)
     {
       new B();
     }
}
```

Invocation of constructor:

```
B() {                    A() {                    Object() {
// super()               //super()                 //load instructions.
//load instructions      //load instructions
// NSI                   // NSI
// User-written          //user written
}                        }                        }
```

Order of Execution (a which constructor gets executed completely)



Output :-

From class A IIB-1

From class A()

From class B IIB-1

From class B()

eg:- class Demo1
{
Demo1()
{
System.out.println("From class Demo1()");
}
{
System.out.println("From class Demo1 IIB-1");
}
}
~~class Demo2 extends Demo1~~
{
Demo2(int i)
{
~~super(10);~~ this();
System.out.println("From class Demo1(int)");
}
}
class Demo2 extends Demo1
{
Demo2()
{
super(10);
System.out.println("From class Demo2()");
}
{
System.out.println("From class Demo2 IIB-1");
}
Demo2(int i)
{
System.out.println("From class Demo2(int)");
}
}
class Driver5
{
public static void main(String[] args)
{

}
}

→ Order of Invocation & order of Execution & o/p

for 1) new Demo2();

2) new Demo2(10);

3) new Demo1(20);

1) new Demo2();

Order of Invocation:

```
Demo2()  ⟶  super Demo1(int i)  ⟶  Demo1()  ⟶  Obj
{              {                    {            {
// super(10)     this();              super().     //
// load instruction                   //load      in
                 //User written        Instru
// NSI.          instruction          //NSI.
//user written                       //user
                                      written
}              }                    }            }
```

Order of Execution:

```
Oblect()  ⟶  Demo1()  ⟶  Demo1(inti)  ⟶  Demo2()
{            {           {              {
             super()
}            }           }              }
```

Output:

From class Demo1 IIB-1

From class Demo1()

From class Demo1(int)

From class Demo2 IIB-1

From class Demo2()

2) new Demo2 (10);

Order of Invocation :

```
Demo2 (int i)  ⟶  Demo1 ( )  ⟶  Object ( )
{                 {              {
super ( )         super ( )      // load instruction
// load instruction  // load inst
// NSI            // NSI
// User written   // user written
}                 }              }
```

Order of Execution :-

```
Object ( )  ⟶  Demo1 ( )  ⟶  Demo2 (int i)
{              {              {
}
               }              }
```

Output :

From class Demo1 IIB-1

From class Demo1 ( )

From class Demo2 IIB-1

From class Demo2 (int)

new Demo1(20) :

Order of Invocation:

```
Demo1(int i) ----> Demo1() ----> Object()
{                  {             {
this();            super()       // load instruction
// UWI;            // NSI        }
                   // UWI
}                  }
```

Order of Execution:

```
Object()  ----> Demo1()  ----> Demo1(int i)
{               {              {
// load instructions  super()     this();
}               //NSI            // UWI;
                // UWI            }
                }
```

output :

From class Demo1 IIB-1

From class Demo1()

From class Demo1(int i)

04/02/2022

NOTE :

```
class super                    class sub extends super
{                              {
super(int i)
{

}

}                              }
```

In the above program we have a compile time issue because inside sub class we don't have a

constructor created by the user.

Therefore, compiler will add a default constructor inside which the 1st instruction will be super(), which tries to call the No argument constructor of its super class.

Inside the class super, we don't have no argument constructor created.

Hence, compiler throws an error.

The above problem has following solutions:

solution1: Creating a No-argument constructor inside the class Super

```
class Super
{
super (int i)
 {
 }
super () {
}
}
```

```
class Sub extends Super
{
}
```

SOLUTION 2: Create a constructor inside Sub class which is having a super() trying to call the parameterised constructor of its super class ie. super(int).

```
class Super
{
 super(int i)
 {
 }
}
```

```
class Sub extends Super
{
 Sub() {
     super(10);
 }
}
```

NOTE: To avoid the above situation it is always good programmers practise to have a no-argument constructor inside classes if we are having a parameterized constructor in it.

NOTE: Private members of the class will not be inherited.

Eg:
```
class Base
{
 private static int i=10;
}
class Derived extends Base
{
 public static void main(String[] args)
 {
  System.out.println(i); //CTE
 }
}
```

TYPES OF INHERITANCE:

In java, we have 5 types of Inheritance.

i) Single - Level Inheritance

ii) Mcelti- level Inheritance

iii) Multiple- level Inheritance

iv) Hierarchal Inheritance

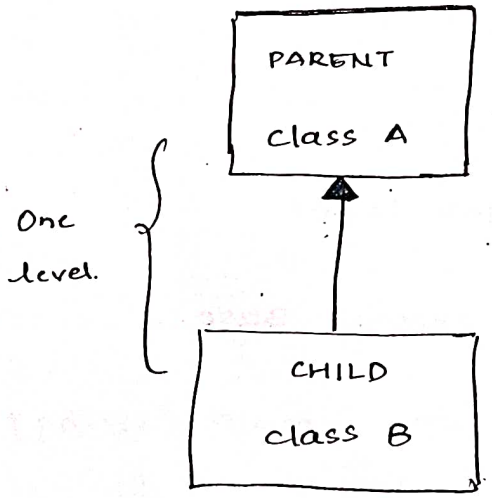
v) Hybrid Inheritance

▶ SINGLE - LEVEL INHERITANCE:

An Inheritance of one level is known as Single- level Inheritance

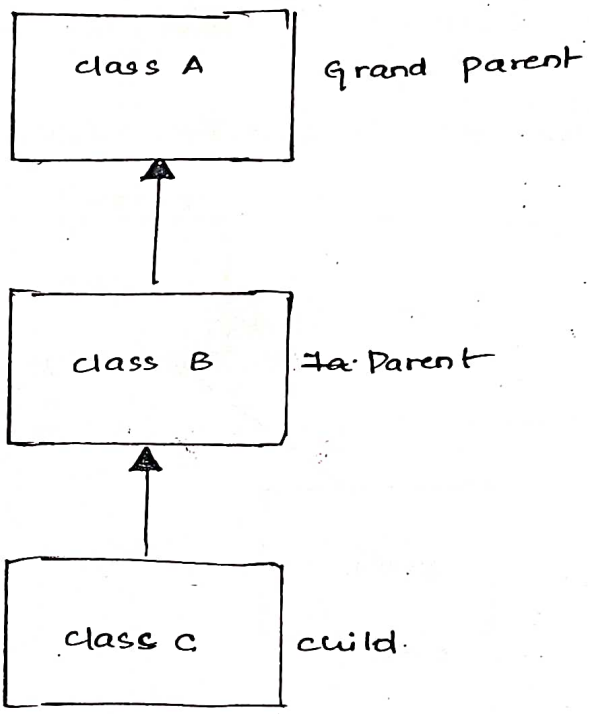
(OR)

Inheritance between one child and one parent is known as Single level Inheritance



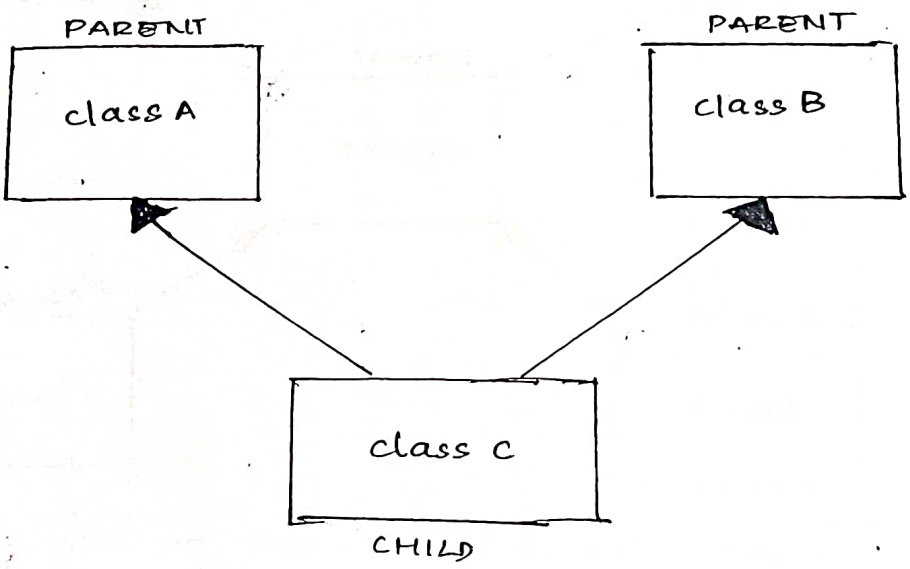
MULTI- LEVEL INHERITANCE :

An Inheritance of more than one level is known as Multi-level Inheritance.



MULTIPLE - LEVEL INHERITANCE :

An Inheritance where one child will have one more ~~one~~ parent in the same level is known as Multiple level Inheritance

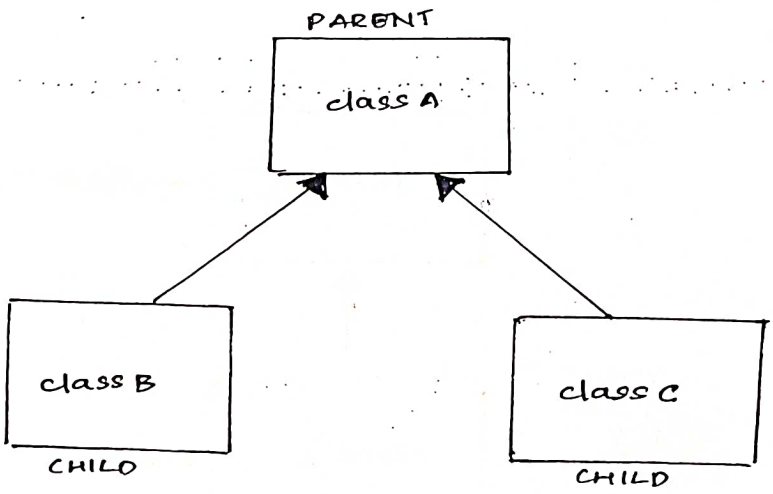


NOTE : Multiple Inheritance is not possible in Java with the help of only class.

ii) It can be achieved with the help of Interface.

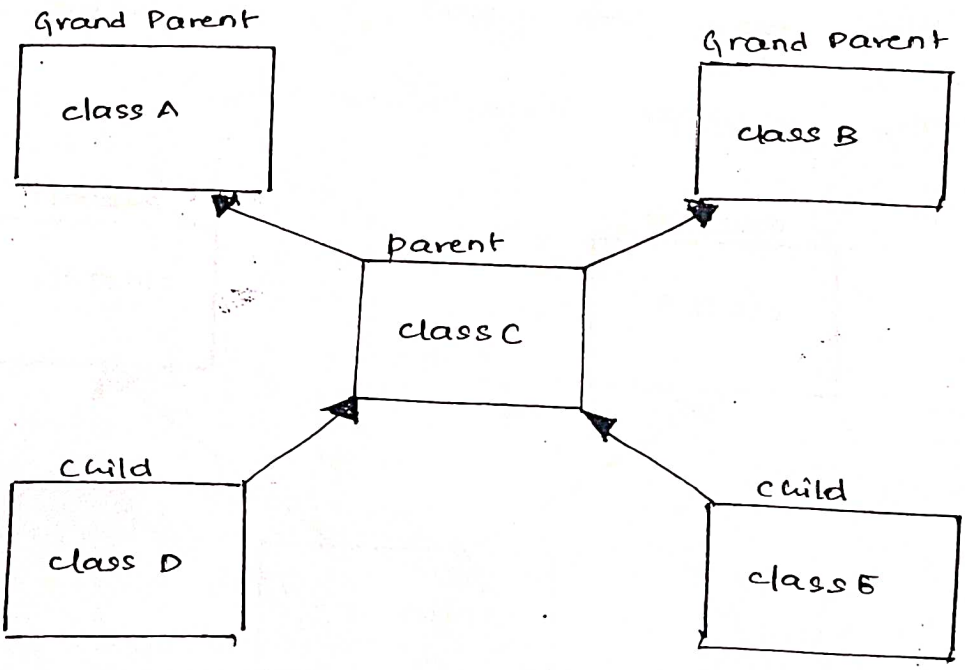
▶ HIERARCHAL INHERITANCE:

An Inheritance where one parent will have more than one child at the the same level is known as heirarchal Inheritance.



▶ HYBRID INHERITANCE:

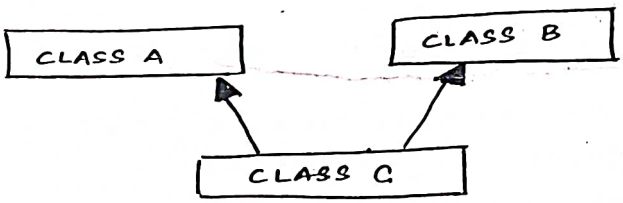
It is a combination of two or more Inheritance



NOTE: Multiple Inheritance is not possible in java with the help of only class because of diamond problem

▶ DIAMOND PROBLEM :

CASE1 :-



Let us consider class C extends to class A & class B. Whenever we create an object of class C during the loading process of class C object, the constructor of class C will try to call the constructor of its super class with the help of super call statement or super().

In this situation there is a dilema or ambiguity whether to call the constructor of cla A or the constructor of class B. Such problem we call it as Diamond problem.

CASE2 : Let us consider class C extends to class A and class B.

Assume both the super classes have method with the same signature.

Whenever we call the method "with the help of the subclass object reference", we will have

an ambiguity to execute either a method of class A or method of class B.

Such problem we call it as Diamond problem

▶ PURPOSE OF super() :-

i) It is used to call constructor of super class from the sub class

ii) It is used to initialize the states of super class from sub class.

Eg:
```
class Accounts
{
 int acno;
 String name;
 double bal;
 String ifsc;
 Accounts()
 {
 }
 Accounts( int acno, String name, double bal, Strin
 {
  this.acno= acno;
  this.name= name;
  this.bal= bal;
  this.ifsc= ifsc;
 }
```

```
public  void  displayDetails()
{
System.out.println("Account number:" + acno);
System.out.println("Account name:" + name);
System.out.println("Account bal:" + bal);
System.out.println("IFSC code of branch:"+ ifsc);
}
}

class  LA  extends  Accounts
{
 double  loanamt;
 double  emi;
 LA()
 {
 }
 LA(int acno, string name , double bal, string ifsc,
      double loanamt, double emi)
 {
    super(acno, name, bal, ifsc);
    this.loanamt = loanamt;
    this.emi = emi;
 }
public  void  displayDetails()
{
    System.out.println("Account number :" + acno);
    System.out.println("Account name:"+ name);
```